

# INFORME TÉCNICO: DESARROLLO DE ANALIZADORES SINTÁCTICOS Y EVALUACIÓN DE RENDIMIENTO

## 1. INTRODUCCIÓN

El presente informe detalla la implementación de diversos sistemas de análisis sintáctico (parsing) desarrollados mediante herramientas de generación automática como ANTLR4 y codificación manual de algoritmos clásicos. El objetivo principal fue comprender la estructura de los lenguajes, la resolución de ambigüedades y la comparativa de eficiencia entre diferentes métodos de procesamiento de tokens.

## 2. DESARROLLO POR PUNTOS

### PUNTOS 1 Y 2: DISEÑO E IMPLEMENTACIÓN DE GRAMÁTICA NOSQL

Se llevó a cabo el diseño de una gramática específica para el reconocimiento de consultas NoSQL. El proceso incluyó:

- Definición BNF: Se estableció la estructura formal del lenguaje en formato Backus-Naur para definir las reglas de las operaciones no relacionales.
- Generación con ANTLR4: Se utilizó el archivo `NoSQL.g4` para generar automáticamente los componentes del Lexer y Parser en lenguaje Python.
- Implementación de Visitor: Se desarrolló un archivo `visitor_NoSQL.py` encargado de recorrer el árbol sintáctico (AST), permitiendo no solo validar la sintaxis, sino interpretar la lógica de las consultas ingresadas.

### PUNTO 3: RESOLUCIÓN DE AMBIGÜEDAD

Se analizó el problema de la ambigüedad en sentencias condicionales anidadas. Para resolverlo:

- Se identificó que una misma sentencia `if-then-else` podía generar múltiples árboles sintácticos.
- Se aplicó una técnica de factorización y reestructuración gramatical, creando reglas diferenciadas para proposiciones emparejadas y no emparejadas.
- El resultado fue una gramática determinista que asocia cada cláusula `else` de forma unívoca al `if` más cercano.

## PUNTO 4: BENCHMARK DE RENDIMIENTO (ANTLR VS. CYK)

Se realizó una comparativa técnica entre un parser predictivo moderno y el algoritmo de programación dinámica CYK:

- ANTLR4: Demostró una eficiencia superior con una complejidad temporal casi lineal  $O(n)$ , manteniendo tiempos de ejecución bajos incluso ante entradas extensas.
- Algoritmo CYK: Se implementó de forma manual, evidenciando un costo computacional cúbico  $O(n^3)$ .
- Resultado: Las pruebas de estrés confirmaron que, mientras ANTLR procesa tokens en tiempo constante, el tiempo de CYK se dispara exponencialmente ante cadenas largas, validando la superioridad de los parsers predictivos para entornos de producción.

## PUNTO 5: ANALIZADOR DESCENDENTE RECURSIVO MANUAL

Se diseñó e implementó desde cero un algoritmo de emparejamiento (match) para una gramática de asignaciones y condicionales:

- Se codificaron funciones recursivas para reglas como **Asignacion** e **if-then**.
- El sistema utiliza un puntero de posición para consumir tokens de manera secuencial.
- Se implementó un manejo de errores sintácticos personalizado que indica exactamente qué token se esperaba y cuál se encontró, garantizando la integridad del análisis.

## 3. CONCLUSIONES

El proyecto permitió validar que la eficiencia de un compilador depende directamente de la estructura de su gramática. La eliminación de la recursividad por la izquierda y la correcta gestión de la ambigüedad son pasos críticos para el éxito del análisis. Finalmente, se concluye que herramientas como ANTLR4 simplifican drásticamente el desarrollo de lenguajes complejos, superando en rendimiento a métodos clásicos de reconocimiento manual cuando se trabaja con volúmenes de datos considerables.